SOURCE LOG: src\ao_optim_monolith.py

```
"""
================================================================================
[illegible]: ao_optim_monolith_v02.py
================================================================================
[illegible] [illegible] TORCHAO [illegible] [illegible] [illegible] [illegible] V02_STABLE_PLUS.
[illegible] [illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] 100%.

[illegible] [illegible] [illegible] [illegible]:
- [illegible] 38 [illegible] LoRA-[illegible] Flux-[illegible].
- [illegible] [illegible] AdamW, [illegible] [illegible] [illegible] [illegible] [illegible] float32
  [illegible] [illegible] int8-[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]. [illegible].
- [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] VRAM [illegible] 2-3 [illegible], [illegible] [illegible]
  [illegible] [illegible] [illegible] [illegible] RTX 3090.
- [illegible] step() [illegible] [illegible] [illegible] int8-[illegible] [illegible] float32 [illegible] [illegible] [illegible]
  [illegible] [illegible] [illegible] [illegible] [illegible], [illegible] [illegible] [illegible] [illegible] [illegible] 21.0 [illegible] VRAM.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]-[illegible] [illegible] [illegible]:
1. [illegible] [illegible] [illegible] (from ao_optim_monolith import ...) [illegible] [illegible]!
   [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] train_engine_v02.py.
2. [illegible] [illegible] [illegible] sys.path [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] src/.
3. [illegible] [illegible] [illegible] [illegible] [illegible], [illegible] TODO, pass [illegible] [illegible].
   [illegible] [illegible] [illegible] [illegible] [illegible], [illegible] [illegible] [illegible] [illegible]!

© 2026 [illegible] [illegible] [illegible] Flowch. [illegible] [illegible] [illegible] [illegible].
================================================================================
"""
import torch
from torch.optim import Optimizer

# [illegible] [illegible] [illegible] [illegible], [illegible] [illegible] [illegible] [illegible]...


import torch
from torch.optim import Optimizer


class OptimState8BitTensor(torch.Tensor):
    """
    [illegible] [illegible]-subclass [illegible] [illegible] [illegible] [illegible] [illegible] 8-[illegible] [illegible].
    [illegible] [illegible] [illegible], [illegible] [illegible] [illegible].
    """
    @staticmethod
    def __new__(cls, elem, scale):
        return torch.Tensor._make_subclass(cls, elem, elem.requires_grad)

    def __init__(self, elem, scale):
        self.scale = scale

    def dequantize(self):
        return self.to(torch.float32) * self.scale

    def __repr__(self):
        return f"OptimState8BitTensor(shape={self.shape}, scale={self.scale})"

def _quantize_8bit(tensor: torch.Tensor):
    """
    [illegible] [illegible] float32 [illegible] [illegible] [illegible] int8 [illegible] [illegible] [illegible].
    [illegible] [illegible] [illegible] AdamW [illegible] [illegible].
    """
    if tensor.device.type != "cuda":
        return tensor, None

    # [illegible] [illegible] [illegible] [illegible]
    max_val = torch.max(torch.abs(tensor))
    if max_val == 0:
        return torch.zeros_like(tensor, dtype=torch.int8), 1.0
```

```
    scale = max_val.item() / 127.0
    quantized = torch.clamp(torch.round(tensor / scale), -127, 127).to(torch.int8)
    return quantized, scale

class AdamW8bit(Optimizer):
    """
    ■■■■■■■■■■■■■ ■■■■■■■■■■■■ 8-■■■■■■■ AdamW ■■ TorchAO.
    ■■■■■■■■ ■■■■■■■■ ■■■■■■ ■■■■■■■■■■■■ VRAM ■■ ■■■■■■■■ Flux ■■ 2-3 ■■.
    """
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=1e-2):
        defaults = dict(lr=lr, betas=betas, eps=eps, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        """■■■■■■■■■■■■ ■■■■■ AdamW ■ 8-■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■■■."""
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            beta1, beta2 = group['betas']
            eps = group['eps']
            lr = group['lr']
            wd = group['weight_decay']

            for p in group['params']:
                if p.grad is None:
                    continue

                grad = p.grad.to(torch.float32)
                state = self.state[p]

                # 1. ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■ (■■■■ 1)
                if len(state) == 0:
                    state['step'] = 0
                    q_avg, s_avg = _quantize_8bit(torch.zeros_like(p))
                    q_sq, s_sq = _quantize_8bit(torch.zeros_like(p))
                    state['exp_avg'] = OptimState8BitTensor(q_avg, s_avg)
                    state['exp_avg_sq'] = OptimState8BitTensor(q_sq, s_sq)

                state['step'] += 1

                # 2. ■■■■■■■■■■■■■■■ ■■■■ ■■■■■■■■■
                exp_avg = state['exp_avg'].dequantize()
                exp_avg_sq = state['exp_avg_sq'].dequantize()

                # 3. ■■■■■■■■■■■■ AdamW
                if wd != 0: p.mul_(1.0 - lr * wd)
                exp_avg.mul_(beta1).add_(grad, alpha=1.0 - beta1)
                exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1.0 - beta2)

                denom = (exp_avg_sq.sqrt() / (1.0 - beta2**state['step'])**0.5).add_(eps)
                p.addcdiv_(exp_avg, denom, value=-(lr / (1.0 - beta1**state['step'])))

                # 4. ■■■■■■■■■ ■■■■■■■■■■■■■■ ■ 8-■■■■
                q_avg, s_avg = _quantize_8bit(exp_avg)
                q_sq, s_sq = _quantize_8bit(exp_avg_sq)
                state['exp_avg'] = OptimState8BitTensor(q_avg, s_avg)
                state['exp_avg_sq'] = OptimState8BitTensor(q_sq, s_sq)

        return loss
```